

图与网络算法及树结构

教学书本式复习讲义（20-30 页版）

按“导言-理论-案例-步骤/代码-结论”结构整理

2026 年 7 月 8 日

使用说明

这份讲义把图论与网络算法、组合优化方法、常见平衡树和编码树放在同一条主线上。每个主题都尽量按照教材写法组织：先说明“解决什么问题”，再给出关键思想、理论依据、简单例题、可执行步骤或 Python 参考实现，最后说明适用条件与常见错误。篇幅有限，所以对证明采用“考试可复述”的深度；对最大流、最小费用最大流、A*、Johnson、红黑树等细节较多的方法，主要给出核心定理和实现框架。

编译建议：使用 XeLaTeX 编译本文件。本文使用 ctexbook、TikZ、booktabs、longtable 与 listings。中文排版采用 ctex，图示采用 TikZ，表格采用 booktabs/longtable，代码采用 listings。

目录

第一章 图模型、遍历与有向无环图	1
1.1 图模型的基本语言	1
1.2 DFS 与 BFS	1
1.2.1 导言与理论	1
1.2.2 Python 参考实现	3
1.3 拓扑排序与关键路径法	3
1.3.1 拓扑排序	3
1.3.2 关键路径法 CPM	3
第二章 最短路算法族	5
2.1 Dijkstra 算法	5
2.1.1 导言	5
2.1.2 核心理论	5
2.2 Bellman-Ford 算法	6
2.3 Floyd-Warshall 算法	7
2.4 Johnson 算法	7
2.5 A* 搜索	7
第三章 最小生成树、网络流与匹配	8
3.1 最小生成树: Kruskal 与 Prim	8
3.1.1 问题定义	8
3.1.2 Kruskal	8
3.1.3 Prim	9
3.2 最大流与最小割	9
3.2.1 模型	9
3.3 最小费用最大流	10
3.4 二分图匹配	10

目录	iii
第四章 组合优化：TSP、图着色与启发式方法	12
4.1 旅行商问题 TSP	12
4.1.1 问题与边界	12
4.2 图着色	12
4.3 遗传算法 GA	13
4.4 模拟退火 SA	13
4.5 蚁群算法 ACO	13
第五章 树结构：B 树、B+ 树、红黑树与赫夫曼树	15
5.1 B 树	15
5.1.1 导言	15
5.1.2 查找、插入、分裂	15
5.2 B+ 树	16
5.3 红黑树	16
5.3.1 旋转的直觉	16
5.4 赫夫曼树	16
第六章 全局比较、建模套路与复习结论	19
6.1 方法总表	19
6.2 建模套路	20
6.3 最后结论	20
参考来源与延伸阅读	21

第一章 图模型、遍历与有向无环图

1.1 图模型的基本语言

定义 1.1 (图、路径与权). 图可记为 $G = (V, E)$, 其中 V 是顶点集, E 是边集。若边有方向, 则称有向图; 若每条边 (u, v) 还有数值 $w(u, v)$, 则称带权图。路径是顶点序列 $v_0, v_1, \dots, v_k$, 且相邻顶点之间有边连接; 路径长度可按边数计算, 也可按权值和计算。

图算法的第一步不是写代码, 而是判断问题属于哪一类模型:

- 只问“能不能到达、是否连通、有没有环”, 通常从 DFS/BFS 出发;
- 边权全非负, 求单源最短路, 优先考虑 Dijkstra;
- 允许负边但不允许负环, 考虑 Bellman-Ford 或 Johnson;
- 要求连接所有点且总代价最小, 考虑最小生成树;
- 有容量、费用、供需、匹配关系时, 考虑网络流或二分图匹配;
- 遍历所有城市、图着色、排课等常常是 NP-困难问题, 需区分精确算法和启发式方法。

1.2 DFS 与 BFS

1.2.1 导言与理论

深度优先搜索 DFS 像“沿一条路走到底再回退”, 适合发现连通分量、环、拓扑序、割点、桥等结构; 广度优先搜索 BFS 像“逐层扩散”, 适合无权最短路、层次遍历和二分图判定。两者在邻接表上都只访问每个顶点和每条边常数次数, 因此复杂度为

$$\mathcal{O}(|V| + |E|).$$

定理 1.1 (BFS 的无权最短路性质). 在无权图中, 从源点 s 进行 BFS, 第一次访问到顶点 v 时得到的层数, 等于从 s 到 v 的最短边数。

证明. BFS 按层扩展: 第 k 层全部由第 $k - 1$ 层顶点一步到达。因此若某点第一次在第 k 层出现, 必存在长度为 k 的路径。若还有更短路径, 则它应在更早层被发现, 矛盾。 $\square$

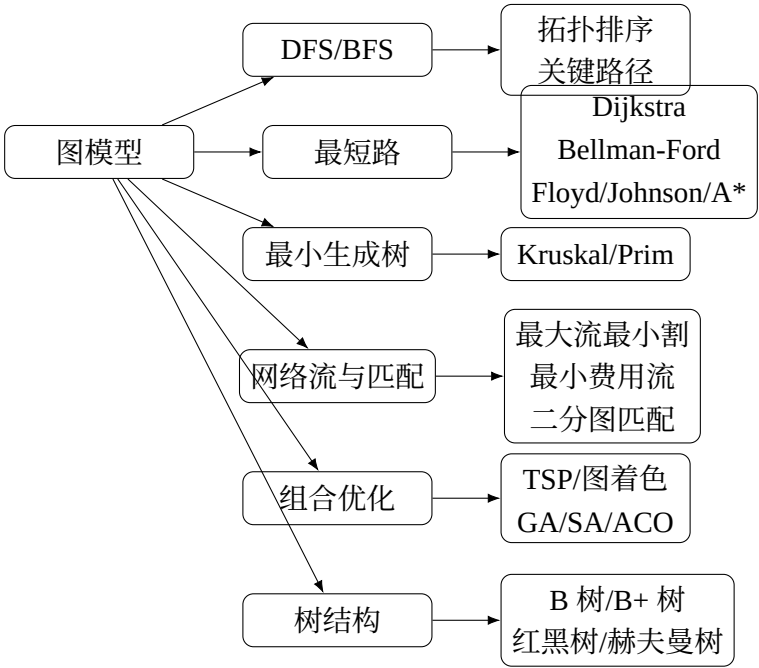


图 1.1: 本讲义的知识主线

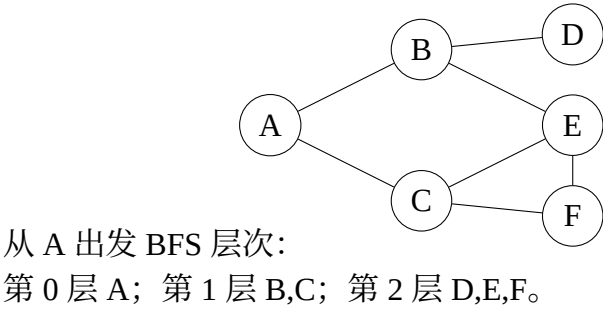


图 1.2: BFS 逐层扩展示意

1.2.2 Python 参考实现

```
1 from collections import deque
2
3 def bfs(graph, s):
4     dist = {s: 0}
5     parent = {s: None}
6     q = deque([s])
7     while q:
8         u = q.popleft()
9         for v in graph.get(u, []):
10             if v not in dist:
11                 dist[v] = dist[u] + 1
12                 parent[v] = u
13                 q.append(v)
14     return dist, parent
15
16 def dfs(graph, s):
17     seen, order = set(), []
18     def visit(u):
19         seen.add(u)
20         order.append(u)
21         for v in graph.get(u, []):
22             if v not in seen:
23                 visit(v)
24     visit(s)
25     return order
```

1.3 拓扑排序与关键路径法

1.3.1 拓扑排序

拓扑排序只适用于有向无环图 DAG。若边 $u \rightarrow v$ 表示“任务 u 必须先于任务 v ”，则拓扑序就是一种合法执行顺序。

Kahn 算法的思想是：反复取入度为 0 的点输出，并删除它的出边。若最后输出点数少于 $|V|$ ，说明图中有环。

$$\text{复杂度} = \mathcal{O}(|V| + |E|).$$

1.3.2 关键路径法 CPM

关键路径法把项目看成 DAG，边或点表示活动，权值表示活动持续时间。求项目总工期，就是求从开始点到结束点的最长路径；在 DAG 中，最长路可按拓扑序动态规划。

设活动网络中 $ve[v]$ 表示事件 v 的最早发生时间, 则

$$ve[v] = \max_{(u,v) \in E} \{ve[u] + w(u, v)\}.$$

反向计算最迟发生时间 $vl[v]$ 后, 活动 (u, v) 的时间余量为

$$slack(u, v) = vl[v] - ve[u] - w(u, v).$$

余量为 0 的活动就是关键活动, 所有关键活动形成的路径称为关键路径。

例 1.1 (简单项目网络). 若活动 $A \rightarrow B$ 需 3 天, $A \rightarrow C$ 需 2 天, $B \rightarrow D$ 需 4 天, $C \rightarrow D$ 需 6 天, 则两条路径长度分别为 7 与 8, 故关键路径为 $A \rightarrow C \rightarrow D$, 项目最短工期为 8 天。

本节要点 DFS 偏结构分析, BFS 偏层次和无权最短路; 拓扑排序必须要求 DAG; 关键路径法本质上是在 DAG 上求最长路。考试中经常要求: 写遍历序、判断有无环、给拓扑序、算最早/最迟时间和关键活动。

第二章 最短路算法族

2.1 Dijkstra 算法

2.1.1 引言

Dijkstra 算法解决非负权图上的单源最短路径问题。给定源点 s ，求它到所有顶点的最短距离。它是典型的贪心算法：每一步把当前暂定距离最小的顶点“定型”。

2.1.2 核心理论

定理 2.1 (Dijkstra 定型正确性). 若所有边权非负，则每次从未定型顶点中选出 $d[u]$ 最小的顶点 u 时，有 $d[u] = \delta(s, u)$ 。

证明. 反设存在一条更短的 $s \rightsquigarrow u$ 路径。在这条路径上取第一个未定型顶点 x ，其前驱 y 已定型。由于 y 定型时会松弛边 (y, x) ，可得 $d[x]$ 不大于真实最短距离。又因边权非负，从 x 到 u 不会减少长度，所以 $d[x] < d[u]$ ，与 u 是未定型点中距离最小者矛盾。 $\square$

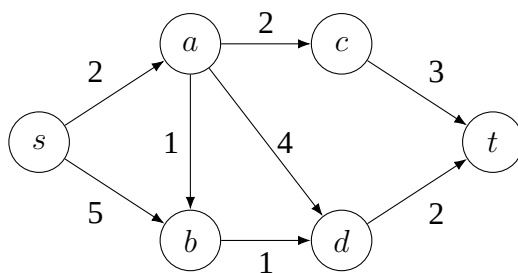


图 2.1: Dijkstra 示例图

从 s 出发，距离变化为：初始 $d[s] = 0$ ；选 s 后 $d[a] = 2, d[b] = 5$ ；选 a 后 $d[b] = 3, d[c] = 4, d[d] = 6$ ；选 b 后 $d[d] = 4$ ；选 c 后 $d[t] = 7$ ；选 d 后 $d[t] = 6$ 。最终最短路 $s \rightarrow a \rightarrow b \rightarrow d \rightarrow t$ ，长度为 6。

```
1 import heapq
2 from math import inf
3
4 def dijkstra(graph, source):
5     dist = {v: inf for v in graph}
```

```

6     prev = {v: None for v in graph}
7     dist[source] = 0
8     heap = [(0, source)]
9     while heap:
10         du, u = heapq.heappop(heap)
11         if du != dist[u]:
12             continue
13         for v, w in graph[u]:
14             if dist[v] > du + w:
15                 dist[v] = du + w
16                 prev[v] = u
17                 heapq.heappush(heap, (dist[v], v))
18     return dist, prev

```

邻接表加二叉堆的常见复杂度为 $\mathcal{O}((V + E) \log V)$ 。注意：只要出现负权边，就不能直接使用 Dijkstra。

2.2 Bellman-Ford 算法

Bellman-Ford 的核心是反复松弛所有边。经过第 k 轮后，所有“边数不超过 k 的最短路”都已被正确考虑。由于没有负环时，简单最短路最多包含 $|V| - 1$ 条边，所以做 $|V| - 1$ 轮即可。

定理 2.2. 若从源点可达的部分不存在负权回路，则 *Bellman-Ford* 在 $|V| - 1$ 轮松弛后得到所有最短距离；若第 $|V|$ 轮仍能松弛某条边，则存在源点可达负环。

```

1 def bellman_ford(vertices, edges, source):
2     INF = 10**18
3     dist = {v: INF for v in vertices}
4     dist[source] = 0
5     for _ in range(len(vertices) - 1):
6         changed = False
7         for u, v, w in edges:
8             if dist[u] != INF and dist[v] > dist[u] + w:
9                 dist[v] = dist[u] + w
10                changed = True
11         if not changed:
12             break
13     for u, v, w in edges:
14         if dist[u] != INF and dist[v] > dist[u] + w:
15             raise ValueError("negative cycle reachable")
16     return dist

```

复杂度为 $\mathcal{O}(VE)$ ，优点是能处理负边并检测负环。

2.3 Floyd-Warshall 算法

Floyd-Warshall 是全网最短路动态规划。设 $d_k[i][j]$ 表示从 i 到 j 的路径中，只允许使用编号不超过 k 的顶点作为中间点时的最短距离，则转移为

$$d_k[i][j] = \min\{d_{k-1}[i][j], d_{k-1}[i][k] + d_{k-1}[k][j]\}.$$

实现时可把三维数组压成二维数组，外层枚举中间点 k ：

```

1 def floyd_warshall(dist):
2     # dist[i][j] is initial edge weight, INF if no edge
3     n = len(dist)
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if dist[i][j] > dist[i][k] + dist[k][j]:
8                     dist[i][j] = dist[i][k] + dist[k][j]
9     return dist

```

复杂度为 $\mathcal{O}(V^3)$ ，适合点数不大、图较稠密、需要任意两点最短路场景。

2.4 Johnson 算法

Johnson 用于稀疏图全源最短路，允许负边但不允许负环。它的关键是“重新赋权”：先加超级源点 q ，用 Bellman-Ford 求势函数 $h(v) = \delta(q, v)$ ，再把每条边权改成

$$w'(u, v) = w(u, v) + h(u) - h(v).$$

由三角不等式可得 $w'(u, v) \geq 0$ ，所以之后可以从每个源点运行 Dijkstra。重赋权不会改变路径之间的相对大小，因为任一路径 $p: s \rightsquigarrow t$ 有

$$w'(p) = w(p) + h(s) - h(t),$$

对固定的 s, t ，后半部分是常数。

2.5 A* 搜索

A* 可看成带启发函数的 Dijkstra。每个候选点用

$$f(n) = g(n) + h(n)$$

排序，其中 $g(n)$ 是起点到当前点的已知代价， $h(n)$ 是当前点到目标点的估计代价。若 h 从不高估真实距离，称为 admissible；若还满足三角不等式形式，称为 consistent。启发函数越接近真实距离，搜索越少；若 $h \equiv 0$ ，A* 退化为 Dijkstra。

本节要点 最短路选择口诀：无权图用 BFS；非负权单源用 Dijkstra；有负边单源用 Bellman-Ford；点数较少的全源用 Floyd-Warshall；稀疏图全源且可有负边用 Johnson；有明确目标并有好启发函数时用 A*。

第三章 最小生成树、网络流与匹配

3.1 最小生成树：Kruskal 与 Prim

3.1.1 问题定义

在连通无向带权图中，生成树连接所有顶点且无环。最小生成树 MST 是权值和最小的生成树。

引理 3.1 (割性质). 对任意割 $(S, V \setminus S)$ ，跨越该割的最小权边必属于某棵最小生成树。

证明. 设 e 是跨割最小边，任取一棵 MST T 。若 $e \in T$ 则成立。若 $e \notin T$ ，把 e 加入 T 会形成一个环，环上必有另一条跨越该割的边 f 。由 $w(e) \leq w(f)$ ，用 e 替换 f 后仍是生成树且总权不增，因此存在包含 e 的 MST。 $\square$

3.1.2 Kruskal

Kruskal 把所有边按权升序排列，从小到大尝试加入，只要不形成环就接受。判环用并查集。

```
1 class DSU:
2     def __init__(self, n):
3         self.parent = list(range(n))
4         self.rank = [0] * n
5     def find(self, x):
6         while self.parent[x] != x:
7             self.parent[x] = self.parent[self.parent[x]]
8             x = self.parent[x]
9         return x
10    def union(self, a, b):
11        ra, rb = self.find(a), self.find(b)
12        if ra == rb:
13            return False
14        if self.rank[ra] < self.rank[rb]:
15            ra, rb = rb, ra
16        self.parent[rb] = ra
17        if self.rank[ra] == self.rank[rb]:
18            self.rank[ra] += 1
19        return True
```

```

20
21 def kruskal(n, edges):
22     dsu = DSU(n)
23     ans = []
24     for w, u, v in sorted(edges):
25         if dsu.union(u, v):
26             ans.append((u, v, w))
27     return ans

```

复杂度主要由排序决定，为 $\mathcal{O}(E \log E)$ 。Kruskal 更适合稀疏图。

3.1.3 Prim

Prim 从一个顶点开始，不断选择“当前树到外部顶点”的最小边。它与 Dijkstra 形式相似，但含义不同：Dijkstra 维护源点到顶点的路径长度，Prim 维护顶点接入当前生成树的最便宜边。

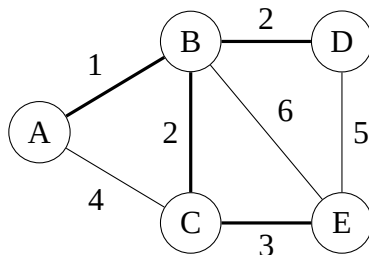


图 3.1: 粗线表示一棵可能的最小生成树

3.2 最大流与最小割

3.2.1 模型

网络流给定源点 s 、汇点 t 和容量 $c(u, v)$ 。流量 $f(u, v)$ 需满足容量限制 $0 \leq f(u, v) \leq c(u, v)$ 与流量守恒。目标是最大化从 s 到 t 的总流量。

割 (S, T) 满足 $s \in S, t \in T$ ，割容量为所有从 S 指向 T 的边容量和：

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

定理 3.1 (最大流最小割定理). 任一网络中，最大 $s-t$ 流的值等于最小 $s-t$ 割的容量。

增广路方法维护残量网络：若残量网络中还有从 s 到 t 的路，就沿路增加流量。Edmonds-Karp 规定每次用 BFS 找最短增广路，复杂度为 $\mathcal{O}(VE^2)$ ；Dinic 用分层图和阻塞流，实践中更快。

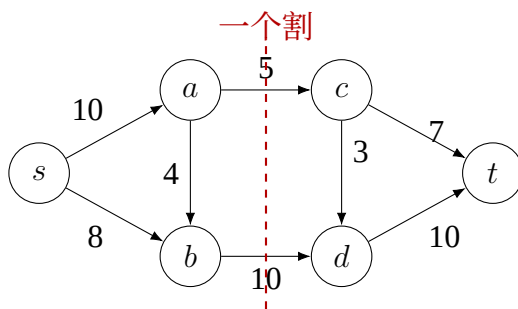


图 3.2: 网络流与割的直观关系

3.3 最小费用最大流

最小费用最大流在每条边上同时有容量 $c(u, v)$ 与单位费用 $a(u, v)$ 。目标是在最大流量的前提下，使总费用

$$\sum_{(u,v) \in E} a(u, v) f(u, v)$$

最小。常见入门做法是“逐次最短增广路”SSAP：每次在残量网络中找一条从 s 到 t 的最小费用路径，再沿路增广。若有负费用边，常结合势能函数把边权调整为非负，以便使用 Dijkstra。

方法步骤：

1. 建立残量网络，并为每条正向边添加反向边；
2. 在残量网络中求从 s 到 t 的最短费用路；
3. 沿该路增广可行流量；
4. 更新残量容量与反向边费用；
5. 直到无法增广，或达到指定流量。

应用包括运输、任务分配、供需平衡、带费用的匹配等。实现时最容易错的是反向边费用、势能更新和负费用处理。

3.4 二分图匹配

二分图 $G = (L \cup R, E)$ 的边只连接左部点与右部点。匹配是一组两两不共享端点的边。最大匹配可以转化为最大流：从源点连到所有左部点，容量为 1；左部点到右部点按原边连接，容量为 1；右部点连到汇点，容量为 1。

Hopcroft-Karp 算法比逐条增广更快。它每一轮先 BFS 建立“最短增广路层次图”，再 DFS 同时寻找一组点不相交的最短增广路，从而把复杂度降到

$$\mathcal{O}(E\sqrt{V}).$$

本节要点 MST 的核心是割性质：Kruskal 按边从小到大，Prim 按顶点逐步扩张。最大流的核心是残量网络，定理终点是最大流等于最小割。最小费用最大流是在流量基础上叠加费用优化。二分图匹配既可用最大流建模，也可用 Hopcroft-Karp 高效求解。

第四章 组合优化：TSP、图着色与启发式方法

4.1 旅行商问题 TSP

4.1.1 问题与边界

TSP 要求从一个城市出发，经过每个城市恰好一次后回到起点，并使总路程最短。它是经典 NP-困难问题。小规模可用动态规划精确求解，大规模通常用启发式或近似方法。

Held-Karp 动态规划设 $dp[S][j]$ 表示从起点 0 出发，访问集合 S 且最后停在 j 的最短距离：

$$dp[S][j] = \min_{i \in S, i \neq j} \{dp[S \setminus \{j\}][i] + d(i, j)\}.$$

最终答案为

$$\min_{j \neq 0} \{dp[V][j] + d(j, 0)\}.$$

复杂度约为 $\mathcal{O}(n^2 2^n)$ ，只能用于较小 n 。

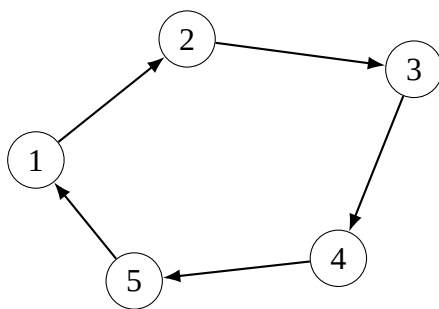


图 4.1: TSP 巡回路径示意

4.2 图着色

图着色要求给每个顶点分配颜色，使相邻顶点颜色不同；最少颜色数称为色数 $\chi(G)$ 。应用包括排课、寄存器分配、无线频率分配。一般图最优着色是难问题，因此常用贪心、回溯、DSATUR 等方法。

贪心着色步骤：按照某种顶点顺序，依次给当前顶点选择最小可用颜色。它快，但不保证最优。DSATUR 的思想是优先处理“饱和度”高的顶点，即邻居中已经出现颜色种类多的顶点。

4.3 遗传算法 GA

遗传算法把一个候选解编码为染色体。对 TSP，可把城市排列作为染色体。典型流程如下：

1. 初始化若干随机个体；
2. 计算适应度，例如路径越短适应度越高；
3. 选择较优个体作为父代；
4. 交叉生成子代；
5. 小概率变异以增加多样性；
6. 重复迭代并保留最好解。

GA 的优点是模型灵活，缺点是参数敏感、收敛不稳定，不能保证全局最优。

4.4 模拟退火 SA

模拟退火从一个解出发，在邻域中随机移动。若新解更好则接受；若更差，也以概率

$$P = \exp\left(-\frac{\Delta}{T}\right)$$

接受，其中 Δ 是变差的幅度， T 是温度。温度高时敢于跳出局部最优，温度低时趋向稳定收敛。

用于 TSP 时，常用 2-opt 作为邻域：选两条边断开并反转中间路径。模拟退火的关键不是公式，而是温度初值、降温速度、每个温度下的迭代次数。

4.5 蚁群算法 ACO

蚁群算法模拟蚂蚁利用信息素寻找路径。对 TSP，蚂蚁从一个城市出发，按“信息素强度”和“启发式距离”选择下一个城市。常见转移概率为

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum_{k \in \text{allowed}} \tau_{ik}^{\alpha} \eta_{ik}^{\beta}},$$

其中 τ_{ij} 是信息素， $\eta_{ij} = 1/d_{ij}$ 是启发信息。每轮后，信息素挥发并在较优路径上增加。

表 4.1: 组合优化方法对比

方法	结果性质	优点	局限
Held-Karp	精确最优	理论清晰，小规模可靠	指数复杂度，规模稍大不可用
贪心着色	近似/启发式	简单快速	强依赖顶点顺序，不保最优
遗传算法	启发式	适合编码清晰的大规模问题	参数敏感，可能早熟
模拟退火	启发式	能跳出局部最优, 容易实现	降温策略影响很大
蚁群算法	启发式	适合路径类组合优化	信息素参数多，易陷入早熟

本节要点 TSP 与图着色要先判断“是否必须最优”。若规模小，可用动态规划或回溯；若规模大，通常选 2-opt、GA、SA、ACO 等启发式。启发式的正确说法是“给出较好可行解”，不要把它说成保证最优。

第五章 树结构：B 树、B+ 树、红黑树与赫夫曼树

5.1 B 树

5.1.1 引言

B 树是面向外存和数据库索引的多路平衡搜索树。二叉搜索树每个节点最多两个孩子，若数据在磁盘中，树高过大就会造成大量 I/O。B 树让一个节点保存多个关键字与多个孩子，使树的高度显著降低。

一棵 m 阶 B 树通常满足：

- 每个节点最多有 m 个孩子；
- 除根外的内部节点至少有 $\lceil m/2 \rceil$ 个孩子；
- 含 k 个孩子的节点有 $k - 1$ 个关键字；
- 所有叶子在同一层。

5.1.2 查找、插入、分裂

查找与普通搜索树类似：在节点内部二分查找关键字区间，再进入对应孩子。插入时先插入叶节点；若节点关键字过多，就把中间关键字上移，左右两部分分裂成两个节点。分裂可能向上传播，甚至产生新根。

B 树：内部节点保存关键字，也可保存记录指针

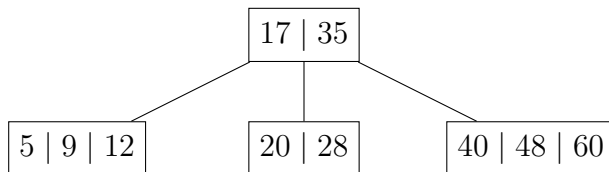


图 5.1: B 树页结构示意图

5.2 B+ 树

B+ 树是数据库索引中更常见的结构。与 B 树相比，B+ 树通常让内部节点只保存索引关键字，不保存完整记录；所有实际记录或记录指针集中在叶节点，叶节点之间还按顺序相连。因此，B+ 树非常适合范围查询：找到范围起点后，沿叶子链表向右扫描即可。

B+ 树：内部节点导航，叶节点链表支持范围查询

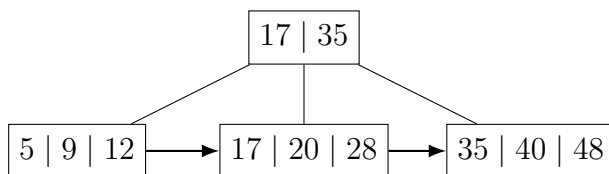


图 5.2: B+ 树索引与叶子链表

5.3 红黑树

红黑树是自平衡二叉搜索树。它给每个节点增加红/黑颜色，并通过旋转与变色维持近似平衡。常用性质如下：

1. 每个节点为红色或黑色；
2. 根节点为黑色；
3. 所有空叶子 NIL 视为黑色；
4. 红色节点不能有红色孩子；
5. 从任一节点到其所有后代 NIL 叶子的简单路径，包含相同数量的黑节点。

这些性质保证树高为 $\mathcal{O}(\log n)$ ，因此查找、插入、删除均为 $\mathcal{O}(\log n)$ 。

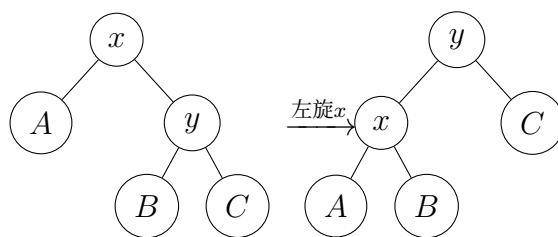
5.3.1 旋转的直觉

旋转不破坏二叉搜索树的中序顺序，只改变局部高度。左旋可理解为“让右孩子上升，当前节点下降到其左侧”；右旋相反。

插入修复的基本思路：新节点先染红；若父亲是黑色则结束；若父亲与叔叔都是红色，则父叔变黑、祖父变红并继续向上；若叔叔是黑色，则通过旋转把“连续红色”修复掉。

5.4 赫夫曼树

赫夫曼树用于构造最优前缀编码。前缀编码要求任一字符编码都不是另一字符编码的前缀，这样解码时不会产生歧义。若字符频率不同，高频字符应使用短编码，低频字符使用长编码。

图 5.3: 红黑树左旋保持中序顺序: $A < x < B < y < C$

赫夫曼算法步骤:

1. 把每个字符看成一棵权为频率的单节点树;
2. 每次取权最小的两棵树合并, 生成新树, 权为二者之和;
3. 重复直到只剩一棵树;
4. 左边记 0, 右边记 1, 即可得到编码。

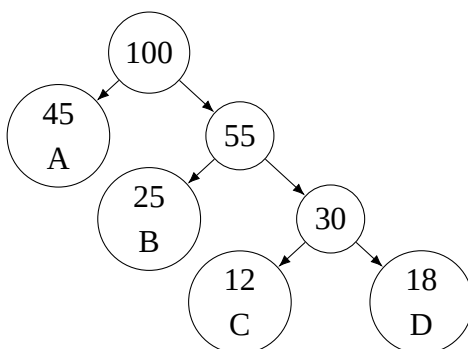


图 5.4: 赫夫曼树示意: 权值越大离根越近

```

1 import heapq
2 from itertools import count
3
4 def huffman(freq):
5     uid = count()
6     heap = [(w, next(uid), ch) for ch, w in freq.items()]
7     heapq.heapify(heap)
8     while len(heap) > 1:
9         w1, _, t1 = heapq.heappop(heap)
10        w2, _, t2 = heapq.heappop(heap)
11        heapq.heappush(heap, (w1 + w2, next(uid), (t1, t2)))
12    tree = heap[0][2]
13    codes = {}
14    def walk(t, code):
15        if isinstance(t, tuple):
16            walk(t[0], code + "0")
17            walk(t[1], code + "1")

```

```
18         else:
19             codes[t] = code or "0"
20         walk(tree, "")
21     return codes
```

本节要点 B 树和 B+ 树是外存索引结构，重点是降低树高和减少磁盘 I/O；红黑树是内存中的有序字典结构，重点是用颜色和旋转保持对数高度；赫夫曼树是编码树，重点是“每次合并最小权”的贪心最优性。

第六章 全局比较、建模套路与复习结论

6.1 方法总表

表 6.1: 图论、网络算法与树结构总览

方法	典型复杂度	适用条件	关键结论/限制
DFS	$\mathcal{O}(V + E)$	图遍历、连通性、环检测	深递归可能爆栈，可改显式栈
BFS	$\mathcal{O}(V + E)$	无权最短路、层次扩展	不适用于带权最短路
拓扑排序	$\mathcal{O}(V + E)$	DAG 依赖关系	有环则不存在拓扑序
关键路径	$\mathcal{O}(V + E)$	项目网络 DAG	本质是 DAG 最长路
Dijkstra	$\mathcal{O}((V + E) \log V)$	非负权单源最短路	负权边会破坏贪心
Bellman-Ford	$\mathcal{O}(VE)$	可含负边	可检测负环，速度较慢
Floyd-Warshall	$\mathcal{O}(V^3)$	全源最短路	适合点数较小或稠密图
Johnson	约 $\mathcal{O}(VE \log V)$	稀疏图全源，可有负边	不能有负环，依赖重赋权
A*	依赖启发函数	有目标点路径搜索	启发函数越好，搜索越少
Kruskal	$\mathcal{O}(E \log E)$	无向连通图 MST	稀疏图常用，需并查集
Prim	$\mathcal{O}(E \log V)$	无向连通图 MST	稠密图可用矩阵版
最大流	Edmonds-Karp $\mathcal{O}(VE^2)$	容量网络	最大流值等于最小割容量
最小费用流	依赖流量与最短路实现	容量加费用网络	注意反向边与负费用
二分图匹配	$\mathcal{O}(E\sqrt{V})$	二分图	可转最大流，也可 Hopcroft-Karp
TSP	$\mathcal{O}(n^2 2^n)$ 精确 DP	小规模精确	大规模通常启发式
图着色	一般难问题	冲突分配、排课	贪心不保最优
GA	迭代式	大规模近似优化	参数敏感，需防早熟
SA	迭代式	局部搜索问题	降温策略关键

方法	典型复杂度	适用条件	关键结论/限制
ACO	迭代式	路径组合优化	信息素参数敏感
B 树/B+ 树	$\mathcal{O}(\log n)$	外存索引	B+ 树范围查询更方便
红黑树	$\mathcal{O}(\log n)$	内存有序映射	旋转与变色维护平衡
赫夫曼树	$\mathcal{O}(n \log n)$	前缀编码	高频短码、低频长码

6.2 建模套路

遇到题目时，可按以下顺序判断：

- 1. 是否有“先后依赖”？若有，先想 DAG、拓扑排序、关键路径。
- 2. 是否要求“最短、最省、最小代价路径”？看边权：无权 BFS，非负 Dijkstra，有负边 Bellman-Ford，全源 Floyd 或 Johnson。
- 3. 是否要求“连接所有点且总成本最小”？用 MST，边排序明显时 Kruskal，点扩张明显时 Prim。
- 4. 是否出现“容量、供给、需求、最多能运多少”？用最大流；若同时出现费用，用最小费用流。
- 5. 是否是“人-任务、课程-教室、男-女、左-右”两类对象配对？优先想到二分图匹配。
- 6. 是否要求“访问所有点一次、排课不冲突、最少颜色”？通常是难问题，要说明精确算法规模限制与启发式方案。
- 7. 是否是“数据库索引、磁盘页、范围查询”？考虑 B 树/B+ 树；若是内存有序字典，考虑红黑树；若是压缩编码，考虑赫夫曼树。

6.3 最后结论

这些算法不是孤立记忆的。它们可以按三条线串起来：第一条是“搜索线”，从 DFS/BFS 到拓扑排序、最短路和 A*；第二条是“优化线”，从 MST 到最大流、最小费用流、匹配、TSP 和图着色；第三条是“数据结构线”，从多路外存索引到内存平衡树，再到编码树。复习时应抓住每个方法的三个问题：它解决什么模型，成立条件是什么，为什么这个步骤正确。代码只是把这些思想落地。

参考来源与延伸阅读

本文主要依据经典论文、算法词典与 LaTeX 官方资料整理。下列链接适合继续查阅原始定义、历史来源和排版工具文档。

1. E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, 1959. <https://link.springer.com/article/10.1007/BF01386390>
2. L. R. Ford and D. R. Fulkerson, *Maximal Flow Through a Network*, 1956. https://www.cs.yale.edu/homes/lans/readings/routing/ford-max_flow-1956.pdf
3. J. E. Hopcroft and R. M. Karp, *An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs*, 1973. <https://epubs.siam.org/doi/10.1137/0202019>
4. P. E. Hart, N. J. Nilsson and B. Raphael, *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*, 1968. <https://ieeexplore.ieee.org/document/4082128/>
5. D. A. Huffman, *A Method for the Construction of Minimum-Redundancy Codes*, 1952. <https://ieeexplore.ieee.org/document/4051119>
6. R. Bayer and E. M. McCreight, *Organization and Maintenance of Large Ordered Indexes*, 1972. <https://link.springer.com/article/10.1007/BF00288683>
7. L. J. Guibas and R. Sedgwick, *A Dichromatic Framework for Balanced Trees*, 1978. <https://sedgewick.io/wp-content/themes/sedgewick/papers/1978Dichromatic.pdf>
8. NIST Dictionary of Algorithms and Data Structures. <https://www.nist.gov/dads/>
9. CTAN ctex: LaTeX classes and packages for Chinese typesetting. <https://ctan.org/pkg/ctex?lang=en>
10. CTAN PGF/TikZ: Create PostScript and PDF graphics in TeX. <https://ctan.org/pkg/pgf>
11. CTAN booktabs: Publication quality tables in LaTeX. <https://ctan.org/pkg/booktabs?lang=en>